

RESIDÊNCIA FULLSTACK



Apostila de Java para Iniciantes

Semana 2 — Fundamentos de Java e Programação Orientada a Objetos

Para quem nunca programou em Java ou está no início.
Do zero até um exercício prático: uma calculadora com POO.



Apostila de Java para Iniciantes

Residência Fullstack — Semana 2

Para quem é essa apostila? Para quem nunca programou em Java ou está no início. Vamos do zero, com exemplos práticos do nosso próprio challenge!

Sumário

- | | |
|-----------------------------------|---|
| 1. O que é Java? | 7. Estruturas de Decisão (if/else) |
| 2. Configurando o Ambiente | 8. Estruturas de Repetição (Loops) |
| 3. Primeiro Programa: Hello World | 9. Arrays e Listas |
| 4. Variáveis e Tipos de Dados | 10. Métodos (Funções) |
| 5. Operadores | 11. Programação Orientada a Objetos (POO) |
| 6. Entrada de Dados com Scanner | 12. 🎯 Exercício: Calculadora com POO |
-

1. O que é Java?

Java é uma linguagem de programação criada pela Sun Microsystems em 1995 e mantida atualmente pela Oracle. É uma das linguagens mais usadas no mundo, especialmente em:

- 🏦 Sistemas bancários e financeiros
- 🏛️ Sistemas do governo
- 📱 Aplicativos Android (até alguns anos atrás)
- 🌐 Backends de grandes empresas (Netflix, LinkedIn, Amazon)

Por que Java?

Característica	O que significa na prática
Fortemente tipada	Você precisa dizer o tipo de cada variável
Orientada a objetos	Organiza o código em "objetos" do mundo real
Multiplataforma	Roda em Windows, Mac e Linux sem mudança
Robusta	Detecta erros antes mesmo de executar
Muito usada no mercado	Fácil de encontrar emprego com Java

Como Java funciona?

```
Você escreve código.java → Compilador Java (javac) → Bytecode (.class) (linguagem JVM) → JVM executa em qualquer SO
```

2. Configurando o Ambiente

Passo 1: Instalar o JDK (Java Development Kit)

1. Acesse: <https://adoptium.net> (versão gratuita e recomendada)
2. Baixe o **JDK 17 ou 21** para seu sistema operacional
3. Instale normalmente

Passo 2: Verificar a instalação

Abra o terminal e digite:

```
java --version
javac --version
```

Se aparecer a versão, está instalado corretamente! 

Passo 3: Escolher uma IDE

IDE	Para quem?	Link
IntelliJ IDEA Community	Recomendada para iniciantes	jetbrains.com/idea
VS Code + Extension Pack for Java	Se já usa o VS Code	code.visualstudio.com
Eclipse	Clássica, muito usada em empresas	eclipse.org

3. Primeiro Programa: Hello World

Todo programador começa aqui. Crie um arquivo chamado `HelloWorld.java` :

```
// Arquivo: HelloWorld.java

public class HelloWorld {           // Todo código Java fica dentro de uma classe

    public static void main(String[] args) { // Ponto de entrada do programa
        System.out.println("Olá, Mundo!"); // Imprime no terminal
    }

}
```

Para executar no terminal:

```
javac HelloWorld.java    # compila: gera HelloWorld.class
java HelloWorld          # executa: roda o programa
```

O que cada parte significa?

Parte	Significado
<code>public class HelloWorld</code>	Declara uma classe pública chamada HelloWorld
<code>public static void main</code>	O método principal — é por aqui que o Java começa
<code>String[] args</code>	Argumentos que podem ser passados pelo terminal
<code>System.out.println(...)</code>	Imprime uma linha no terminal

💡 **Regra importante:** o nome do arquivo `.java` deve ser **idêntico** ao nome da classe.
`HelloWorld.java` → `class HelloWorld`.

4. Variáveis e Tipos de Dados

Uma **variável** é uma caixinha com nome que guarda um valor.

Em Java, você precisa sempre dizer **que tipo** de dado vai guardar:

```
// Números inteiros (sem vírgula)
int idade = 20;
long populacaoBrasil = 215000000L; // Long para números muito grandes

// Números com vírgula (use ponto, não vírgula!)
double altura = 1.75;
float nota = 8.5f;

// Texto (String com S maiúsculo!)
String nome = "João";
String mensagem = "Olá, mundo!";

// Caractere único
char letra = 'A';

// Verdadeiro ou falso
boolean aprovado = true;
boolean reprovado = false;
```

Dicas importantes

```
// ❌ Errado: não pode mudar o tipo
int numero = 10;
numero = "dez"; // ERRO! numero é int, não String

// ✅ Certo: podemos mudar o valor mas não o tipo
int numero = 10;
numero = 20; // OK!

// Constantes: valores que não mudam (use maiúsculas por convenção)
```

```
final int MAXIMO_TENTATIVAS = 3;
final double PI = 3.14159;
```

Conversão entre tipos

```
// String para número
String texto = "42";
int numero = Integer.parseInt(texto);    // String → int
double decimal = Double.parseDouble("3.14"); // String → double

// Número para String
int valor = 100;
String valorTexto = String.valueOf(valor); // int → String
// ou simplesmente: String valorTexto = "" + valor;
```

5. Operadores

Aritméticos

```
int a = 10;
int b = 3;

System.out.println(a + b);    // 13  (soma)
System.out.println(a - b);    // 7   (subtração)
System.out.println(a * b);    // 30  (multiplicação)
System.out.println(a / b);    // 3   (divisão inteira – atenção!)
System.out.println(a % b);    // 1   (resto da divisão – módulo)

// Para divisão com casas decimais, use double:
double resultado = (double) a / b; // 3.3333...
```

Comparação (retornam **true** ou **false**)

```
int x = 5;
System.out.println(x == 5);    // true  (igual a)
System.out.println(x != 3);    // true  (diferente de)
System.out.println(x > 3);     // true  (maior que)
System.out.println(x < 3);     // false (menor que)
```

```
System.out.println(x >= 5); // true (maior ou igual)
System.out.println(x <= 4); // false (menor ou igual)
```

Lógicos (combinam condições)

```
boolean a = true;
boolean b = false;

System.out.println(a && b); // false (E – ambos precisam ser true)
System.out.println(a || b); // true (OU – pelo menos um true)
System.out.println(!a); // false (NÃO – inverte o valor)

// Exemplo prático:
int idade = 20;
boolean temCNH = true;

boolean podeDigir = (idade >= 18) && temCNH;
System.out.println(podeDigir); // true
```

6. Entrada de Dados com Scanner

Para ler o que o usuário digita no terminal, usamos a classe `Scanner` :

```
import java.util.Scanner; // Importação obrigatória!

public class EntradaDados {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in); // Cria o Scanner

        // Ler uma String
        System.out.print("Digite seu nome: ");
        String nome = scanner.nextLine();

        // Ler um número inteiro
        System.out.print("Digite sua idade: ");
        int idade = scanner.nextInt();
        scanner.nextLine(); // ← IMPORTANTE: limpa a quebra de linha após nextInt()

        // Ler um número decimal
        System.out.print("Digite sua altura (ex: 1.75): ");
        double altura = scanner.nextDouble();
```

```

        System.out.println("Olá, " + nome + "!");
        System.out.println("Você tem " + idade + " anos e " + altura + "m de altura.")

        scanner.close(); // Boa prática: fechar o scanner ao final
    }
}

```

⚠ **Armadilha comum:** depois de `nextInt()` ou `nextDouble()`, sempre chame `nextLine()` para limpar a quebra de linha pendente, antes de ler uma String com `nextLine()`.

7. Estruturas de Decisão (if/else)

O `if/else` permite que seu programa tome decisões:

```

int nota = 7;

// Estrutura básica
if (nota >= 7) {
    System.out.println("Aprovado! ✅");
} else {
    System.out.println("Reprovado. ❌");
}

```

Múltiplas condições com else if

```

int score = 75; // Score do nosso detector de golpes

if (score > 60) {
    System.out.println("🚨 PERIGO – Provável golpe!");
} else if (score >= 20) {
    System.out.println("⚠ SUSPEITO – Fique alerta.");
} else {
    System.out.println("✅ SEGURO – Mensagem parece legítima.");
}

```

Switch (quando há muitas opções fixas)

```
String dia = "segunda";

switch (dia) {
    case "segunda":
    case "terça":
    case "quarta":
    case "quinta":
    case "sexta":
        System.out.println("Dia de trabalho.");
        break;
    case "sábado":
    case "domingo":
        System.out.println("Fim de semana! 🎉");
        break;
    default:
        System.out.println("Dia inválido.");
}
```

Operador ternário (if/else em uma linha)

```
int idade = 18;
String status = (idade >= 18) ? "maior de idade" : "menor de idade";
System.out.println("Você é " + status);
```

8. Estruturas de Repetição (Loops)

For — quando sabemos quantas vezes repetir

```
// Conta de 1 até 5
for (int i = 1; i <= 5; i++) {
    System.out.println("Contando: " + i);
}
// Saída: Contando: 1, Contando: 2, ... Contando: 5

// For em array (percorre todos os elementos)
String[] nomes = {"Ana", "Bruno", "Carlos"};
for (String nome : nomes) { // Para cada nome no array:
```

```
System.out.println("Olá, " + nome + "!");  
}
```

While — enquanto uma condição for verdadeira

```
int tentativas = 0;  
int senhaCorreta = 1234;  
int senhaDigitada = 0;  
  
Scanner scanner = new Scanner(System.in);  
  
while (senhaDigitada != senhaCorreta && tentativas < 3) {  
    System.out.print("Digite a senha: ");  
    senhaDigitada = scanner.nextInt();  
    tentativas++;  
}  
  
if (senhaDigitada == senhaCorreta) {  
    System.out.println("Acesso liberado! ✓");  
} else {  
    System.out.println("Conta bloqueada! ✗");  
}
```

Do-While — executa pelo menos uma vez

```
String resposta;  
Scanner scanner = new Scanner(System.in);  
  
do {  
    System.out.print("Deseja continuar? (s/n): ");  
    resposta = scanner.nextLine();  
} while (!resposta.equalsIgnoreCase("s") && !resposta.equalsIgnoreCase("n"));  
  
System.out.println("Você escolheu: " + resposta);
```

9. Arrays e Listas

Arrays (tamanho fixo)

```
// Declaração
int[] numeros = {10, 20, 30, 40, 50};
String[] frutas = new String[3]; // Array vazio de 3 posições

// Acessar elementos (índice começa em 0!)
System.out.println(numeros[0]); // 10 (primeiro)
System.out.println(numeros[4]); // 50 (último)

frutas[0] = "Maçã";
frutas[1] = "Banana";
frutas[2] = "Uva";

// Tamanho do array
System.out.println(numeros.length); // 5
```

ArrayList (tamanho dinâmico — recomendada!)

```
import java.util.ArrayList;
import java.util.List;

List<String> sinais = new ArrayList<>(); // Lista vazia de Strings

// Adicionar
sinais.add("Urgência detectada");
sinais.add("Link suspeito");
sinais.add("Pedido de dados");

// Verificar tamanho
System.out.println(sinais.size()); // 3

// Percorrer
for (String sinal : sinais) {
    System.out.println("→ " + sinal);
}

// Verificar se contém
System.out.println(sinais.contains("Urgência detectada")); // true

// Remover
sinais.remove("Link suspeito");
```

💡 Quando usar qual?

- **Array**: quando você sabe exatamente quantos itens vai ter (ex: dias da semana = 7)
- **ArrayList**: quando o número de itens pode variar (ex: lista de sinais encontrados)

10. Métodos (Funções)

Métodos são blocos de código com nome que realizam uma tarefa. Evitam repetição de código.

Estrutura de um método

```
[modificador] [tipo de retorno] nomeDoMetodo([parâmetros]) {  
    // corpo do método  
    return valor; // se não for void  
}
```

Exemplos

```
public class ExemploMetodos {  
  
    public static void main(String[] args) {  
        // Chamando os métodos  
        saudar("Maria");                // imprime "Olá, Maria!"  
        int soma = somar(10, 5);        // soma = 15  
        boolean ehAdulto = verificarIdade(20); // ehAdulto = true  
  
        System.out.println("Soma: " + soma);  
        System.out.println("É adulto? " + ehAdulto);  
    }  
  
    // Método sem retorno (void): só faz algo, não devolve valor  
    public static void saudar(String nome) {  
        System.out.println("Olá, " + nome + "!");  
    }  
  
    // Método com retorno int: calcula e devolve o resultado  
    public static int somar(int a, int b) {  
        return a + b;  
    }  
}
```

```
// Método com retorno boolean: responde uma pergunta
public static boolean verificarIdade(int idade) {
    return idade >= 18;
}

}
```

Por que usar métodos?

```
// ❌ Sem métodos: código repetido e difícil de manter
System.out.println(10 + 5);
System.out.println(20 + 15);
System.out.println(100 + 200);

// ✅ Com métodos: limpo, organizado e reutilizável
System.out.println(somar(10, 5));
System.out.println(somar(20, 15));
System.out.println(somar(100, 200));
```

11. Programação Orientada a Objetos (POO)

POO é um paradigma (jeito de pensar) onde modelamos o mundo real em **objetos**. Cada objeto tem:

- **Atributos** → características (o que o objeto *é/tem*)
- **Métodos** → comportamentos (o que o objeto *faz*)

Analogia do mundo real

```
Objeto: Carro
├─ Atributos: marca, modelo, cor, velocidade, ligado
└─ Métodos: ligar(), desligar(), acelerar(), frear()

Objeto: Pessoa
├─ Atributos: nome, idade, cpf, email
└─ Métodos: apresentar(), calcularIdade(), validarEmail()
```

Os 4 Pilares da POO

Pilar	Significado simples
Encapsulamento	Esconder os dados internos, expor só o necessário
Herança	Uma classe filha herda características da classe mãe
Polimorfismo	O mesmo método se comporta de formas diferentes
Abstração	Modelar apenas o que é relevante para o problema

Classes e Objetos

Classe = molde/planta **Objeto** = coisa criada a partir do molde

```
// — Definição da CLASSE —————
public class Pessoa {

    // ATRIBUTOS (características) – private = encapsulamento!
    private String nome;
    private int idade;
    private String email;

    // CONSTRUTOR – chamado quando criamos um novo objeto
    public Pessoa(String nome, int idade, String email) {
        this.nome = nome;      // 'this' refere ao atributo da classe
        this.idade = idade;
        this.email = email;
    }

    // GETTERS – métodos para ACESSAR atributos privados
    public String getNome() {
        return nome;
    }

    public int getIdade() {
        return idade;
    }

    // SETTERS – métodos para MODIFICAR atributos com validação
    public void setEmail(String email) {
        if (email.contains("@")) {
            this.email = email;
        } else {
            System.out.println("Email inválido!");
        }
    }
}
```

```

    }
}

// MÉTODO de comportamento
public void apresentar() {
    System.out.println("Olá! Meu nome é " + nome + " e tenho " + idade + " anos.")
}

public boolean ehAdulto() {
    return idade >= 18;
}
}

// — Uso da CLASSE (criação de OBJETOS) —
public class Principal {

    public static void main(String[] args) {

        // 'new' cria um novo objeto (instância) da classe Pessoa
        Pessoa pessoa1 = new Pessoa("Ana", 25, "ana@gmail.com");
        Pessoa pessoa2 = new Pessoa("Carlos", 16, "carlos@gmail.com");

        // Chamar métodos dos objetos
        pessoa1.apresentar(); // "Olá! Meu nome é Ana e tenho 25 anos."
        pessoa2.apresentar(); // "Olá! Meu nome é Carlos e tenho 16 anos."

        System.out.println(pessoa1.getNome() + " é adulta? " + pessoa1.ehAdulto()); //
        System.out.println(pessoa2.getNome() + " é adulta? " + pessoa2.ehAdulto()); //

        // Tentar modificar o email com validação
        pessoa1.setEmail("email-invalido"); // "Email inválido!"
        pessoa1.setEmail("ana.nova@gmail.com"); // OK!
    }
}

```

Encapsulamento na prática

```

// ❌ Sem encapsulamento — qualquer um pode bagunçar os dados
public int idade; // atributo público
pessoa.idade = -500; // ninguém valida!

// ✅ Com encapsulamento — dados protegidos
private int idade; // atributo privado
public void setIdade(int idade) {
    if (idade > 0 && idade < 150) { // validação no setter
        this.idade = idade;
    }
}
}

```

12. Exercício: Calculadora com POO

Enunciado

Criar uma **Calculadora** em Java usando Programação Orientada a Objetos com as seguintes operações:

- **+** Soma
- **-** Subtração
- **×** Multiplicação
- **÷** Divisão (com tratamento de divisão por zero)

Requisitos:

- Criar uma classe `Calculadora` com métodos para cada operação
- Criar uma classe `Main` (ou `CalculadoraApp`) com o método `main`
- O programa deve rodar no terminal, pedindo dois números e a operação desejada
- Tratar o caso de divisão por zero com mensagem de erro adequada
- Implementar um loop para o usuário fazer quantas contas quiser

Código Completo Comentado

Vamos criar dois arquivos:

Arquivo 1: `Calculadora.java` (a classe com as regras)

```
/**
 * CLASSE CALCULADORA
 * -----
 * Esta classe representa uma calculadora.
 * Ela tem ATRIBUTOS (histórico) e MÉTODOS (as operações).
 *
 * Conceito de POO aplicado:
 * - ENCAPSULAMENTO: atributos private, acesso via métodos
 * - ABSTRAÇÃO: escondemos a implementação, expondo só o necessário
 */
public class Calculadora {

    // — ATRIBUTOS —
    // 'private' = encapsulamento (ninguém acessa diretamente de fora)
```

```

private int totalOperacoes;    // Conta quantas operações foram feitas
private double ultimoResultado; // Guarda o último resultado calculado

// — CONSTRUTOR —————
// É chamado quando fazemos 'new Calculadora()'
// Inicializa os atributos com valores padrão

public Calculadora() {
    this.totalOperacoes = 0;    // Começa sem nenhuma operação
    this.ultimoResultado = 0;    // Resultado inicial = 0
}

// — MÉTODOS DAS OPERAÇÕES —————
// Cada método recebe dois números (parâmetros) e retorna o resultado

/**
 * SOMA: retorna a adição de dois números
 * Exemplo: somar(10, 5) → 15.0
 */
public double somar(double a, double b) {
    double resultado = a + b;    // Realiza a soma
    this.ultimoResultado = resultado; // Guarda o resultado
    this.totalOperacoes++;      // Incrementa o contador
    return resultado;           // Devolve para quem chamou
}

/**
 * SUBTRAÇÃO: retorna a diferença entre dois números
 * Exemplo: subtrair(10, 5) → 5.0
 */
public double subtrair(double a, double b) {
    double resultado = a - b;
    this.ultimoResultado = resultado;
    this.totalOperacoes++;
    return resultado;
}

/**
 * MULTIPLICAÇÃO: retorna o produto de dois números
 * Exemplo: multiplicar(10, 5) → 50.0
 */
public double multiplicar(double a, double b) {
    double resultado = a * b;
    this.ultimoResultado = resultado;
    this.totalOperacoes++;
    return resultado;
}

/**
 * DIVISÃO: retorna o quociente.

```

```

    * ATENÇÃO: Trata o caso de divisão por zero!
    * Divisão por zero é matematicamente indefinida → lançamos exceção.
    * Exemplo: dividir(10, 5) → 2.0
    * Exemplo: dividir(10, 0) → lança ArithmeticException
    */
    public double dividir(double a, double b) {
        // Verifica se o divisor é zero ANTES de dividir
        if (b == 0) {
            // 'throw' lança uma exceção (erro proposital com mensagem)
            throw new ArithmeticException("Erro: Divisão por zero não é permitida!");
        }
        double resultado = a / b;
        this.ultimoResultado = resultado;
        this.totalOperacoes++;
        return resultado;
    }

    // — GETTERS —————
    // Métodos para acessar os atributos privados (encapsulamento)

    public int getTotalOperacoes() {
        return totalOperacoes;
    }

    public double getUltimoResultado() {
        return ultimoResultado;
    }

    // — MÉTODO DE APRESENTAÇÃO —————

    /**
     * Exibe um resumo do histórico da calculadora.
     * Void = não retorna nada, só executa uma ação.
     */
    public void exibirHistorico() {
        System.out.println("\n📋 Histórico da sessão:");
        System.out.println("    Total de operações: " + totalOperacoes);
        System.out.println("    Último resultado:   " + ultimoResultado);
    }
}

```

Arquivo 2: **CalculadoraApp.java** (o programa principal)

```

import java.util.Scanner;

/**
 * CLASSE CALCULADORAAPP – Programa Principal
 * -----
 * Esta é a classe que tem o método 'main' e controla o programa.
 */

```

```
* Aqui USAMOS a classe Calculadora (criamos um OBJETO dela).
*
* Separar a lógica (Calculadora.java) da interface (CalculadoraApp.java)
* é uma boa prática de POO chamada "Separação de Responsabilidades".
*/

public class CalculadoraApp {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in); // Para ler o teclado

        // — Criando o OBJETO Calculadora —————
        // 'new Calculadora()' chama o construtor da classe
        // 'calc' é nossa variável que aponta para esse objeto
        Calculadora calc = new Calculadora();

        // Banner de boas-vindas
        System.out.println("┌───────────────────────────────────────────────────────────────────────────────────┐");
        System.out.println("│                                     📊  CALCULADORA COM POO  📊                                     │");
        System.out.println("│                               Residência Fullstack – Java                               │");
        System.out.println("└───────────────────────────────────────────────────────────────────────────────────┘");

        // — Loop principal —————
        // O programa fica rodando até o usuário escolher sair
        boolean continuar = true;

        while (continuar) {

            System.out.println("\n┌───────────────────────────────────────────────────────────────────────────────────┐");
            System.out.println("Escolha a operação:");
            System.out.println("  1. ➕ Soma");
            System.out.println("  2. ➖ Subtração");
            System.out.println("  3. ✖ Multiplicação");
            System.out.println("  4. ÷ Divisão");
            System.out.println("  5. 📋 Ver histórico");
            System.out.println("  0. 🚪 Sair");
            System.out.print("\nOpção: ");

            // Lê a opção do usuário
            int opcao = scanner.nextInt();
            scanner.nextLine(); // Limpa o buffer (boa prática)

            // — Switch para tratar cada opção —————
            switch (opcao) {

                case 1: // Soma
                    double[] nums = pedirDoisNumeros(scanner);
                    double resultadoSoma = calc.somar(nums[0], nums[1]);
                    // Formatando: %.2f = mostrar 2 casas decimais
                    System.out.printf("✅ Resultado: %.2f + %.2f = %.2f\n",
                        nums[0], nums[1], resultadoSoma);
                }
            }
        }
    }
}
```

```

        break;

    case 2: // Subtração
        double[] numsS = pedirDoisNumeros(scanner);
        double resultadoSub = calc.subtrair(numsS[0], numsS[1]);
        System.out.printf("✅ Resultado: %.2f - %.2f = %.2f%n",
            numsS[0], numsS[1], resultadoSub);
        break;

    case 3: // Multiplicação
        double[] numsM = pedirDoisNumeros(scanner);
        double resultadoMult = calc.multiplicar(numsM[0], numsM[1]);
        System.out.printf("✅ Resultado: %.2f x %.2f = %.2f%n",
            numsM[0], numsM[1], resultadoMult);
        break;

    case 4: // Divisão – tem tratamento de erro!
        double[] numsD = pedirDoisNumeros(scanner);
        // 'try-catch' trata erros (exceções) que podem ocorrer
        try {
            double resultadoDiv = calc.dividir(numsD[0], numsD[1]);
            System.out.printf("✅ Resultado: %.2f ÷ %.2f = %.2f%n",
                numsD[0], numsD[1], resultadoDiv);
        } catch (ArithmeticException e) {
            // Se a divisão por zero ocorrer, cai aqui
            System.out.println("❌ " + e.getMessage());
        }
        break;

    case 5: // Histórico
        // Chama o método do OBJETO calc
        calc.exibirHistorico();
        break;

    case 0: // Sair
        continuar = false; // Sai do while
        calc.exibirHistorico(); // Mostra resumo final
        System.out.println("\n👋 Até logo! Obrigado por usar a calculadora");
        break;

    default: // Opção inválida
        System.out.println("⚠️ Opção inválida! Digite um número de 0 a 5.");
    }
}

scanner.close();
}

/**
 * MÉTODO AUXILIAR: pede dois números ao usuário.
 * Retorna um array com os dois valores.

```

```

    * Separar isso num método evita repetição de código!
    */
    private static double[] pedirDoisNumeros(Scanner scanner) {
        System.out.print("🖨️ Digite o primeiro número: ");
        double num1 = scanner.nextDouble();

        System.out.print("🖨️ Digite o segundo número: ");
        double num2 = scanner.nextDouble();

        scanner.nextLine(); // Limpa o buffer

        return new double[]{num1, num2}; // Retorna os dois números
    }
}

```

🔍 Conceitos de POO Aplicados na Calculadora

Conceito	Onde aparece	Por que foi usado
Classe	<code>Calculadora</code> , <code>CalculadoraApp</code>	Organiza o código em blocos lógicos
Objeto	<code>Calculadora calc = new Calculadora()</code>	Instância real da classe no programa
Construtor	<code>public Calculadora()</code>	Inicializa atributos ao criar o objeto
Encapsulamento	Atributos <code>private</code> + getters	Protege os dados internos
Métodos	<code>somar()</code> , <code>subtrair()</code> , etc.	Comportamentos do objeto
Retorno	<code>return resultado</code>	Devolve o resultado calculado
Parâmetros	<code>(double a, double b)</code>	Dados que o método recebe
Exceção	<code>throw new ArithmeticException(...)</code>	Trata erro de divisão por zero
Try-Catch	No <code>CalculadoraApp</code>	Captura e trata o erro graciosamente

Como executar o exercício

```
# 1. Compilar os dois arquivos de uma vez
javac Calculadora.java CalculadoraApp.java

# 2. Executar o programa principal
java CalculadoraApp
```

Casos de teste para a Calculadora

Operação	Entrada A	Entrada B	Saída esperada
Soma	10	5	15.00
Subtração	10	5	5.00
Multiplicação	10	5	50.00
Divisão	10	5	2.00
Divisão por zero	10	0	Mensagem de erro
Números negativos	-5	3	-2.00 (subtração)
Números decimais	3.5	1.5	5.00 (soma)

Resumo Final — Cheat Sheet Java

```
// — Tipos básicos —
int numero = 42;           // inteiro
double decimal = 3.14;     // decimal
String texto = "Olá";      // texto
boolean flag = true;       // verdadeiro/falso

// — Entrada e saída —
Scanner sc = new Scanner(System.in);
String linha = sc.nextLine(); // lê texto
int num = sc.nextInt(); sc.nextLine(); // lê int
System.out.println("Texto: " + linha);
System.out.printf("Valor: %.2f\n", 3.14);
```

```

// — Decisão —————
if (condição) { } else if (outra) { } else { }

// — Repetição —————
for (int i = 0; i < 10; i++) { }
while (condição) { }
for (String item : lista) { } // for-each

// — Classe / Objeto —————
public class MinhaClasse {
    private int atributo;
    public MinhaClasse(int atributo) { this.atributo = atributo; }
    public int getAtributo() { return atributo; }
    public void fazerAlgo() { System.out.println("Fazendo!"); }
}
MinhaClasse obj = new MinhaClasse(10);
obj.fazerAlgo();

// — ArrayList —————
List<String> lista = new ArrayList<>();
lista.add("item");
lista.size();
lista.contains("item");
for (String s : lista) { }

```

 Apostila produzida para a Residência Fullstack — Semana 2 Qualquer dúvida, procure o Vinicius! 😊